

Architectural Blueprint for Web-Based 3D-to-2D Floor Plan Reconstruction Platforms

Ingestion Engine and Heterogeneous 3D Format Decoding

Constructing an interactive web platform capable of converting heterogeneous 3D assets into clean 2D architectural floor plans requires a decoupled, high-performance ingestion engine. Web browsers enforce stringent computational constraints: execution threads operate in sandboxed memory spaces, garbage collection pauses induce rendering hitches, and device capabilities vary widely between mobile and desktop clients¹. Ingesting assets across common interchange formats—such as glTF/GLB, OBJ, FBX, IFC, and USDZ—demands format-specific parsing pipelines optimized for WebAssembly (WASM) and modern WebGL or WebGPU contexts¹.

The standard runtime representation for web-delivered geometry is the glTF/GLB format, standardized by the Khronos Group. The Three.js GLTFLoader decodes binary glTF data directly into contiguous Float32Array buffers under a permissive MIT license, facilitating near zero-copy streaming of vertex attributes into GPU Vertex Buffer Objects (VBOs). Wavefront OBJ files, parsed via the MIT-licensed OBJLoader, present a text-based polygon structure. While parsing OBJ files is procedurally straightforward, decoding large text files entirely within single-threaded JavaScript introduces substantial garbage collection churn, necessitating off-thread chunked parsing inside Web Workers.

Autodesk FBX represents an intricate binary hierarchy embedding transformation matrices, skeletal animations, and material properties. The Three.js FBXLoader relies on reverse-engineered binary parsing routines that frequently incur runtime overhead when traversing deep node graphs.

In contrast, Industry Foundation Classes (IFC) files represent Building Information Modeling (BIM) data containing parametric architectural objects—such as walls, slabs, windows, and doors—governed by STEP physical file standards³. Rather than treating the model as an unorganized triangle mesh, IFC parsers extract semantic objects directly⁵. Two open-source, commercially permissive libraries lead browser-based IFC decoding: web-ifc, a C++ engine compiled via Emscripten under the Mozilla Public License 2.0 (MPL-2.0)³, and ifc-lite-wasm, a Rust-based, multithreaded WASM implementation also licensed under MPL-2.0². These engines allow client applications to query structural building layers specifically—filtering for instances of IfcWall or IfcWallStandardCase while discarding complex mechanical, electrical, and plumbing data⁴.

The Universal Scene Description format (USDZ), authored primarily for Apple AR ecosystems, encapsulates assets within uncompressed zip containers. Web-based USDZ decoding remains

constrained; while the Three.js USDZLoader handles basic geometries⁸, complete specification coverage requires heavier WebAssembly builds of the Pixar OpenUSD library. To maintain a fluid 60 frames per second interface during the ingestion of any of these formats, parsing routines must execute inside background Web Workers, transferring unpacked geometry to the primary graphics loop via transferable ArrayBuffer objects.

Format	Primary In-Browser Loader	License	Underlying Architecture	Web Viability & Memory Footprint
glTF / GLB	three/addons/loaders/GLTFLoader.js	MIT	Direct TypedArray buffer streaming	Optimal; minimal memory overhead; native to web graphics pipelines.
OBJ	three/addons/loaders/OBJLoader.js	MIT	Procedural string tokenizer	Moderate; causes garbage collection overhead on multi-megabyte files.
FBX	three/addons/loaders/FBXLoader.js	MIT	Binary tree node traversal	Variable; complex scene graphs can stall single-threaded loops.
IFC	web-ifc / ifc-lite-wasm [cite: 2, 3]	MPL-2.0 ²	C++ / Rust compiled to WebAssembly ²	High; extracts semantic building geometry directly into typed arrays ⁶ .

USDZ	three/addons/loaders/USDZLoader.js [cite: 8]	MIT	Zip extraction and USDA/USDC parsing	Constrained; limited support for arbitrary parametric schemas ⁸ .
------	--	-----	--------------------------------------	--

Computational Geometry Pipeline: Mathematical Slicing and Vector Extraction

Deriving accurate 2D floor plans from raw 3D polygon meshes requires a pipeline of deterministic geometric algorithms. Architectural standards dictate that floor plans represent an orthographic cross-section sliced horizontally at sill or eye level, typically 1.0m to 1.2m above the finished floor plane ($z = 1.2\text{m}$). The computational task begins with evaluating the intersection between an infinite cutting plane and every polygon in the mesh.

Planar Cross-Section Evaluation and Spatial Acceleration

The cutting plane Π is mathematically expressed by the implicit linear equation:

$$\mathbf{n} \cdot \mathbf{x} + d = 0$$

where $\mathbf{n}$ is the unit normal vector—oriented along the vertical axis as $[0, 0, 1]^T$ for standard horizontal cuts—and d denotes the signed scalar offset along that axis. For any individual triangle T defined by vertices $(\mathbf{v}_0, \mathbf{v}_1, \mathbf{v}_2)$, the signed distance s_i from each vertex $\mathbf{v}_i$ to the plane is calculated as:

$$s_i = \mathbf{n} \cdot \mathbf{v}_i + d$$

If all three distances evaluate to strictly positive or strictly negative values, the triangle resides entirely to one side of the plane and produces zero intersections. When the signs differ, the cutting plane intersects two edges of the triangle. The precise intersection coordinate $\mathbf{p}$ along the line segment bounded by vertices $\mathbf{v}_a$ and $\mathbf{v}_b$ is resolved through linear interpolation:

$$\mathbf{p} = \mathbf{v}_a + \left(\frac{-s_a}{s_b - s_a} \right) (\mathbf{v}_b - \mathbf{v}_a)$$

Iterating over every triangle across an unindexed mesh of several hundred thousand polygons

incurs an $\mathcal{O}(N)$ computational cost per slicing operation, which degrades interactivity. Real-time evaluation requires spatial partitioning via Bounding Volume Hierarchies (BVH). The open-source library three-mesh-bvh constructs an axis-aligned bounding box (AABB) tree directly over Three.js BufferGeometry structures under an MIT license¹. By traversing the BVH nodes using either the Surface Area Heuristic (SAH) or Spatial Midpoint Partitioning (SMP), non-intersecting sub-branches are culled in $\mathcal{O}(\log N)$ time¹. This spatial acceleration extracts raw intersection line segments within milliseconds, even on complex models¹.

Topological Reconstruction, Boolean Healing, and Regularization

Planar slicing yields an unstructured collection of disjoint 3D line segments. To convert this segment soup into usable 2D vector boundaries, the coordinates are projected onto an affine

coordinate plane (u, v) and assembled into continuous topological contours. Because floating-point rounding errors prevent segment endpoints from matching exactly, vertices

must be quantized to a spatial grid using an epsilon clustering threshold ($\epsilon \approx 10^{-4}\text{m}$). The unified vertices are then linked into directed polyline graphs.

Real-world 3D scans and uncurated CAD assets regularly present non-manifold artifacts, self-intersecting faces, missing interior surfaces, and physical scan clutter. To heal these topological defects, the application passes raw 2D contours to the Clipper2 library via clipper2-ts or clipper2-js, distributed under the Boost Software License 1.0⁹.

Clipper2 operates natively in integer coordinate space, circumventing floating-point arithmetic instability⁹. Segment coordinates are multiplied by a scale factor (10^4) and cast to 64-bit integers⁹. Open paths can be thickened into closed solids via morphological dilation (inflatePaths with Minkowski sums)¹⁰. A subsequent boolean union operation dissolves self-intersections, merges overlapping wall boundaries, and eliminates internal structural voids⁹.

The consolidated boundaries are simplified and rectified to match architectural drafting standards. Initial polyline decimation is performed using the Ramer-Douglas-Peucker (RDP) algorithm, which discards intermediate vertices whose perpendicular distance to the segment

chord falls below a predefined tolerance ϵ_{RDP} :

$$d(\mathbf{p}_k, \overline{\mathbf{p}_i \mathbf{p}_j}) = \frac{|(\mathbf{p}_{j,y} - \mathbf{p}_{i,y})\mathbf{p}_{k,x} - (\mathbf{p}_{j,x} - \mathbf{p}_{i,x})\mathbf{p}_{k,y} + \mathbf{p}_{j,x}\mathbf{p}_{i,y} - \mathbf{p}_{j,y}\mathbf{p}_{i,x}|}{\sqrt{(\mathbf{p}_{j,y} - \mathbf{p}_{i,y})^2 + (\mathbf{p}_{j,x} - \mathbf{p}_{i,x})^2}}$$

Because human-built architecture predominantly adheres to the Manhattan World assumption, wall segments should run parallel or perpendicular to primary reference axes. A regularization

pass calculates the orientation angle $\theta = \text{atan2}(\Delta y, \Delta x)$ for each simplified edge. If the

angle aligns within a narrow threshold ($|\theta - (k \cdot \pi/2)| \leq 5^\circ$, where $k \in \{0, 1, 2, 3\}$), the edge is snapped directly to the nearest cardinal axis. This process rectifies jagged scanning noise and reconstructs clean, perpendicular wall corners.

Geometry Library	License	Primary Architectural Function	Execution Context	Performance Characteristics
three-mesh-bvh [cite: 1]	MIT	Spatial tree acceleration, fast triangle-plane intersection tests ¹	JavaScript / TypeScript	Low CPU memory overhead; rapid tree queries on Three.js buffers ¹ .
manifold-3d [cite: 13, 14]	Apache-2.0	Watertight CSG booleans, robust 3D slicing, and 2D cross-sections ¹³	C++ compiled to WASM ¹³	Guaranteed manifold output; eliminates numerical intersection failures ¹³ .
clipper2-ts [cite: 10]	BSL-1.0 ¹⁰	2D polygon offsetting, union/difference, topological healing ¹⁰	Pure TypeScript / JavaScript	Integer-based stability; prevents precision errors during path buffering ⁹ .
earcut / earclip [cite: 16]	ISC / Apache-2.0	Constrained 2D polygon triangulation for rendering hatched wall cross-sections ¹⁶	JavaScript / WASM ¹⁶	Extremely low latency; cleanly handles degenerate and self-overlapping paths ¹⁶ .
CGAL	GPL /	Exact	Native C++ /	Restrictive;

(Reference)	Commercial	arithmetic kernel, surface reconstruction, alpha shapes	Heavy WASM	copyleft terms require commercial licensing for proprietary distribution.
-------------	------------	---	------------	---

Performance Engineering and the Limits of Polycam Parity

Achieving performance parity with native mobile applications like Polycam in a web application requires evaluating the architectural differences between native mobile runtimes and browser-based sandboxes. Polycam captures spaces using the Apple RoomPlan API, which streams data directly from hardware LiDAR time-of-flight sensors. The RoomPlan system tracks device trajectory in 3D space, matches sensor readings to parametric bounding surfaces, and executes proprietary machine learning models on the Apple Neural Engine in real time. Polycam does not process arbitrary, unindexed polygonal meshes after capture; it samples structured point fields while actively guiding the user to record walls, windows, and doors.

In a web-based platform, the application receives an arbitrary, pre-existing 3D model with no sensory depth cues, camera trajectories, or parametric building metadata. Delivering sub-second conversion requires overcoming JavaScript's single-threaded event loop and dynamic memory management:

Web Workers must be used to decouple heavy geometric computations from the user interface. Slicing high-density meshes directly on the main UI thread interrupts frame rendering, leading to noticeable interface lag. By delegating BVH construction, edge-plane intersection testing, and contour linking to a dedicated Web Worker, the main thread remains responsive. Transferring geometry via transferable ArrayBuffer instances avoids memory duplication between worker threads.

Compiling core computational geometry routines into WebAssembly with 128-bit SIMD (Single Instruction, Multiple Data) execution extensions allows the browser to perform parallel vector calculations. Slicing routines compiled from C++ or Rust using SIMD instructions evaluate multiple vertex-plane distances in a single CPU cycle, improving mathematical execution speeds by a factor of three to six compared to standard JavaScript loops.

For high-density meshes containing over one million triangles, intersection workloads can be delegated directly to the client GPU via WebGPU compute shaders². Triangle data stored in uniform GPU buffers is evaluated across thousands of GPU threads simultaneously. Each thread tests an individual triangle for intersection with the cutting plane and writes output segment coordinates into an output storage buffer. This reduces CPU responsibilities to planar topological chaining and simplification.

Object allocation within inner calculation loops must be minimized. Frequently instantiating temporary classes—such as Three.js Vector3 or Line3 objects—triggers browser garbage

collection cycles that cause frame drops. Slicing engines must rely on pre-allocated, flat TypedArrays that reuse contiguous memory buffers throughout intersection passes.

Architectural Dimension	Native Sensor Pipelines (Polycam)	Web-Based Post-Hoc Slicing Pipeline
Input Data Modality	Active LiDAR time-of-flight sensor streams + visual-inertial odometry	Unstructured 3D polygonal meshes (OBJ, glTF, FBX, IFC) ¹⁸
Hardware Execution Layer	Apple Silicon Neural Engine + Metal framework	Browser WebAssembly runtime (SIMD) + WebGPU Compute ²
Processing Paradigm	Real-time parametric plane-fitting during data capture	Post-hoc computational slicing, topological healing, and polygon extraction
Spatial Awareness	Direct physical scale derived from calibrated hardware sensors	Relative coordinates requiring bounding-box normalization and scale inference
End-to-End Latency	Continuous generation at 60 FPS during recording	Sub-second extraction on static uploads (approx. 200ms–800ms)

Intelligent Rectification: Clarifying the Role of Cursor SDK and Specialized Neural Models

Integrating artificial intelligence can significantly improve the conversion of imperfect 3D models into clean 2D architectural plans. However, developers must distinguish between developer-centric orchestration frameworks and production computer vision models to design an effective system architecture.

Clarifying the Scope of the Cursor SDK

The Cursor SDK (@cursor/sdk), available for TypeScript and Python environments, is a software development kit designed to automate coding agent workflows¹⁹. The SDK enables developers to invoke the autonomous agents powering the Cursor IDE programmatically—allowing them to inspect repositories, execute terminal commands, modify source code, and run custom integration tools via the Model Context Protocol (MCP)¹⁹.

The Cursor SDK is not an image-processing engine, spatial computing runtime, or geometric

inference pipeline¹⁹. Incorporating the Cursor SDK into an end-user web application to slice meshes or reconstruct floor plans represents an architectural mismatch¹⁹.

The SDK provides substantial value, however, within the development and platform orchestration layers:

Developers can use the Cursor SDK to automate code generation pipelines that synthesize parametric 3D models or script Three.js scene graphs based on extracted room measurements¹⁹. In enterprise workflows, the SDK can orchestrate MCP server tools that analyze building models, run automated unit tests against architectural standards, or validate IFC schemas against regulatory building codes²¹. The Cursor SDK serves as a developer productivity and agentic automation tool rather than an in-browser runtime for geometry processing¹⁹.

Specialized Neural Architectures for Floor Plan Vectorization

Inferring building layouts, room topologies, and architectural elements from unorganized point clouds or noisy polygon meshes requires dedicated spatial deep learning models:

RoomFormer introduces a transformer-based framework for single-stage floor plan reconstruction²⁵. The model projects 3D point clouds or dense mesh vertices into top-down, orthographic density maps²⁵. Built upon a Deformable-DETR transformer backbone, RoomFormer applies a two-level query mechanism: the primary query layer detects room polygons, while the secondary query layer predicts corner coordinates for each respective room²⁵. The model jointly optimizes polygon classification and corner localization, directly generating closed vector loops that define distinct functional spaces, doorways, and window openings²⁵.

Raster2Seq formulates architectural reconstruction as an autoregressive sequence generation task²⁷. Taking top-down raster projections of 3D spaces, Raster2Seq utilizes a vision-language encoder paired with a sequence-to-sequence polygon decoder to output vector coordinate sequences²⁷. This approach generalizes across non-standard, curved, and non-Manhattan floor layouts where rigid orthogonal heuristics fall short²⁷.

CAGE (Continuity-Aware edGE Network) incorporates edge-continuity constraints into the network architecture to stabilize floor plan reconstruction²⁸. By coupling edge-detection modules with differentiable polygon decoders, CAGE mitigates vector fragmentation caused by physical obstructions, furniture clutter, or incomplete scan surfaces²⁸.

Deploying these neural models within a commercial web platform requires one of two deployment patterns:

Client-side execution can be achieved by quantizing models like RoomFormer to INT8 representations and serving them through onnxruntime-web using the WebGPU execution provider. This pattern processes data entirely within the client's browser, eliminating recurring GPU cloud infrastructure costs and preserving data privacy.

Alternatively, server-side inference microservices deployed on platforms like AWS ECS, Modal, or RunPod can host full-precision checkpoints (such as Swin-Transformer variants)²⁸. The client web app uploads an orthographic density map derived from the 3D model, the remote microservice runs the neural model, and the server returns a structured JSON payload

containing categorized vector loops and room types.

Model / Framework	Target Architecture	Primary Architectural Task	Operational Context	Licensing / Commercial Profile
RoomFormer [cite: 25]	Deformable-D ETR with two-level queries ²⁵	Direct vector room polygon and corner prediction ²⁵	Server microservice (PyTorch) or ONNX Web	Open research checkpoint (academic origin; check weights license) ²⁵ .
Raster2Seq [cite: 27]	Vision-Language Seq2Seq Transformer ²⁷	Autoregressive floor plan polygon sequence generation ²⁷	Server microservice (PyTorch)	Academic open-source release (SIGGRAPH 2026) ²⁷ .
CAGE [cite: 28]	Edge-continuity transformer network ²⁸	Robust polygon boundary reconstruction under occlusion ²⁸	Server microservice (PyTorch) ²⁸	Open-source research release (NeurIPS 2025) ²⁸ .
ONNX Runtime Web	Cross-platform ML inference runtime	Client-side execution of quantized neural vision backbones	Browser WebGPU / WASM execution provider	MIT License; fully permissive for commercial software.
Cursor SDK [cite: 19]	Autonomous agent orchestration harness ¹⁹	Agentic workflow automation, MCP tooling, code synthesis ¹⁹	Node.js backend environment (not in-browser CV) ¹⁹	Commercial proprietary SDK by Cursor ¹⁹ .

Dual-Canvas Platform Architecture, Serialization, and Deployment

Building an interactive web-based architectural platform requires a decoupled presentation layer that synchronizes 3D perspective manipulation with 2D vector editing and CAD asset serialization.

The Dual-Canvas Synchronization Model

The user interface should separate the 3D perspective viewport from the 2D orthographic editing canvas:

The 3D canvas is powered by Three.js or @react-three/fiber, displaying the complete architectural model with orbit controls, lighting, and an interactive cutting plane gizmo. When a user drags the cutting plane gizmo along the vertical axis, a debounced event triggers the background Web Worker slicing routine.

The 2D canvas is implemented using dedicated 2D graphics libraries such as Konva.js (via react-konva), Pixi.js, or direct SVG DOM nodes. The 2D view renders the regularized vector paths output by the geometry engine. Users can select and adjust individual wall nodes, drag architectural stamps (such as doors and windows), verify automated dimensioning callouts, and assign room labels.

Centralized state synchronization is handled via lightweight stores such as Zustand. The store maintains a unified data model defining wall segment coordinates, openings, and metadata. Updates made in the 2D editor reflect immediately in the 3D viewport by updating an extruded preview mesh, while adjusting the 3D slicing gizmo recalculates the 2D vector lines.

Vector Serialization: SVG and CAD DXF Formats

To integrate with commercial architectural and engineering workflows, the platform must export floor plans to both standard web graphics and CAD formats:

Scalable Vector Graphics (SVG) generation compiles the 2D polyline arrays into an XML string. Outer building boundaries and internal room cutouts are formatted using the SVG even-odd fill rule (fill-rule="evenodd"), allowing the vector plan to cleanly render solid wall fills while preserving interior room voids. Dimension markers, text annotations, and door swing arcs are written to isolated SVG <g> group elements with distinctive class names for downstream styling.

AutoCAD Drawing Exchange Format (DXF) files must be generated using an MIT-licensed library such as @tarikjabiri/dxf (or dxf-writer)²⁹. Professional CAD platforms organize drawings into discrete, standardized layers.

The serialization engine groups entities into distinct layers—such as WALLS, DOORS, WINDOWS, and ANNOTATIONS—with assigned AutoCAD Color Indices (ACI)³¹. Wall paths serialize into LWPOLYLINE or continuous LINE entities, while door openings export with their corresponding swing trajectories³⁰. The resulting DXF text string converts into a downloadable Blob directly within client memory, requiring no server-side compute.

Production Cloud Architecture and Publishing Pipelines

Transforming the application into a shareable platform requires an infrastructure that handles file storage, multi-threaded client execution, and public project distribution:

Static assets—including HTML, JavaScript bundles, CSS, and compiled WebAssembly binaries—are distributed globally using edge content networks such as Cloudflare Pages or Vercel. To permit Web Workers to access SharedArrayBuffer for multithreaded WASM

operations, edge web servers must return specific HTTP security headers:

```
HTTP
Access-Origin-Openner-Policy:same-origin
Access-Origin-Embedder-Policy:require-corp
```

User-uploaded 3D assets and exported production drawings (SVG, DXF) stream directly to S3-compatible, egress-free object storage, such as Cloudflare R2 or AWS S3, using secure presigned URLs. Bypassing the application application servers for large asset uploads reduces bandwidth costs and improves transfer speeds.

Project metadata—including user authentication details, revision histories, floor plan JSON schemas, and access permissions—is managed via a relational PostgreSQL database hosted on Supabase or Neon.

When a user publishes a project, the client saves the synchronized 2D vector schema alongside a compressed 3D representation (such as an optimized binary glTF or compressed IFC fragment) to object storage⁷. The backend generates a unique, unguessable public slug (e.g., app.domain.com/view/p_7x9a2b).

Public viewing routes load a lightweight, read-only viewer bundle that downloads the static assets and renders synchronized 2D and 3D views without loading heavier drafting tools, ensuring fast load times across desktop and mobile devices.

Platform Layer	Selected Technology	License	Architectural Responsibility
Frontend Framework	Next.js (React) / SvelteKit	MIT	User interface, state management, client routing, and SSR hydration.
3D Rendering Core	Three.js / @thatopen/fragments [cite: 7]	MIT	WebGL/WebGPU 3D viewport, scene lighting, and cutting plane gizmos ⁷ .
BIM/IFC Extraction	ifc-lite-wasm / web-ifc [cite: 2, 7]	MPL-2.0 ²	Parsing parametric architectural structures and isolating wall/floor

			entities ⁴ .
Spatial Indexing	three-mesh-bvh [cite: 1]	MIT	Rapid triangle-plane intersection testing and AABB acceleration ¹ .
Geometry Kernel	manifold-3d (WASM) ¹³	Apache-2.0	Watertight CSG booleans, slicing, and 2D cross-section generation ¹³ .
2D Vector Healing	clipper2-ts [cite: 10]	BSL-1.0 ¹⁰	Offsetting paths, calculating boolean unions, and resolving self-intersections ¹⁰ .
2D Plan Canvas	Konva.js (react-konva)	MIT	Interactive 2D canvas, vector node editing, snapping, and dimensioning.
CAD Export Engine	@tarikjabiri/dxf [cite: 30, 31]	MIT ³¹	Serializing vector boundaries into layered AutoCAD DXF drawings ³⁰ .
Deep Learning Inference	ONNX Runtime Web / PyTorch Microservice	MIT / Apache-2.0	Executing RoomFormer/Raster2Seq models to vectorize scans and classify spaces ²⁷ .
Developer Agent Tooling	Cursor SDK (@cursor/sdk) ¹⁹	Proprietary / Commercial ¹⁹	Driving internal agentic workflows, code generation, and MCP tooling ¹⁹ .

Cloud Storage	Cloudflare R2 / AWS S3	Commercial Cloud	Storing multi-format 3D files and generated SVG/DXF assets with zero egress costs.
Edge Compute	Cloudflare Workers / Vercel Edge	Commercial Cloud	Serving public slugs, setting COOP/COEP headers, and proxying API requests.

Architectural Conclusions

Building a web-based platform that transforms diverse 3D models into accurate 2D floor plans requires coordinating distinct computational layers rather than relying on a single processing library.

Heterogeneous 3D file ingestion must separate standard triangle meshes (glTF, OBJ) from parametric BIM schemas (IFC)⁵. While glTF streams directly into GPU memory, IFC assets require WebAssembly parsers like web-ifc or ifc-lite-wasm to extract native wall elements and discard extraneous data².

Interactive slicing performance requires spatial acceleration. Developers should use

three-mesh-bvh to reduce triangle-plane intersection tests from $\mathcal{O}(N)$ to $\mathcal{O}(\log N)$, execute computational loops inside background Web Workers using SIMD-enabled WebAssembly kernels (such as manifold-3d), and apply clipper2-ts to resolve topological non-manifold defects within a stable integer coordinate space¹. This multi-threaded architecture delivers sub-second cross-section extraction on standard consumer hardware. Artificial intelligence should be deployed selectively based on model capabilities. Agent orchestration tools like the Cursor SDK belong in developer automation and parametric script synthesis workflows rather than in-browser geometric parsing¹⁹. To infer missing structural boundaries or classify room semantics from noisy scans, teams should deploy specialized vision transformers—such as RoomFormer or Raster2Seq—via server-side microservices or client-side WebGPU runtimes via onnxruntime-web²⁵.

Finally, the web platform should maintain a dual-canvas architecture that synchronizes a 3D WebGL/WebGPU viewport with an interactive 2D vector editor. Exporting finalized plans to layered AutoCAD DXF files using @tarikjabiri/dxf alongside standard Scalable Vector Graphics provides full interoperability with professional engineering and architectural workflows³⁰.

Deployed to edge CDNs with strict cross-origin isolation and backed by egress-free object storage, this architecture provides a scalable foundation for modern browser-based spatial

computing.

Works cited

1. Web-based BVH-accelerated 3D cross-section clipping via graph, <https://www.tandfonline.com/doi/full/10.1080/20964471.2026.2644727>
2. IFCLite. Open, view, and work with IFC files. Right in the browser., <https://ifclite.dev/>
3. GitHub - ThatOpen/engine_web-ifc: Reading and writing IFC files, https://github.com/thatopen/engine_web-ifc
4. IFC Viewer Online — Free Browser-Based BIM Viewer, Validator, <https://www.ifcvieweronline.eu/>
5. IFC Viewer - GitHub, <https://github.com/davras5/ifc-viewer>
6. ifc-lite-wasm - WebAssembly - Lib.rs, <https://lib.rs/crates/ifc-lite-wasm>
7. Getting started - Introduction | That Open docs, <https://docs.thatopen.com/fragments/getting-started>
8. three.js webgl - loaders - USDZLoader - Code for FUKUI, https://code4fukui.github.io/three.js_examples/webgl_loader_usdz.html
9. clipper2-js CDN by jsDelivr - A CDN for npm and GitHub, <https://www.jsdelivr.com/package/npm/clipper2-js>
10. countertype/clipper2-ts - GitHub, <https://github.com/countertype/clipper2-ts>
11. geoclipper2 0.0.1 on npm - Libraries.io - security & maintenance, <https://libraries.io/npm/geoclipper2>
12. JohnClarking/CoolEngines - GitHub, <https://github.com/JohnClarking/CoolEngines>
13. manifold3d - PyPI, <https://pypi.org/project/manifold3d/>
14. manifold3d - PyPI, <https://pypi.org/project/manifold3d/2.5.1/>
15. Bitbybit Docs - TypeScript API Reference, <https://docs.bitbybit.dev/>
16. earclip — Rust implementation // Lib.rs, <https://lib.rs/crates/earclip>
17. csgrs - crates.io: Rust Package Registry, <https://crates.io/crates/csgrs>
18. ifc · GitHub Topics, <https://github.com/topics/ifc?l=javascript>
19. Cursor TypeScript SDK | Cursor Documentation, <https://cursor.com/docs/sdk/typescript>
20. Cursor Python SDK | Cursor Documentation, <https://cursor.com/docs/sdk/python>
21. Cursor SDK June 2026: Custom Tools, Stores, Auto-Review - Nextdev, <https://www.joinnextdev.com/blog/cursor-sdk-june-2026-custom-tools-stores-auto-review>
22. Introducing the Cursor Python SDK - Release Discussions, <https://forum.cursor.com/t/introducing-the-cursor-python-sdk/161367>
23. threers — Rust game dev // Lib.rs, <https://lib.rs/crates/threers>
24. Build programmatic agents with the Cursor SDK, <https://cursor.com/blog/typescript-sdk>
25. [CVPR 2023] RoomFormer: Two-level Queries for Single-stage, <https://github.com/ywyue/roomformer>
26. Floorplan Reconstruction Using Two-Level Queries - Yuanwen Yue,

- <https://ywyue.github.io/RoomFormer/>
27. Raster2Seq: Polygon Sequence Generation for Floorplan ... - GitHub,
<https://github.com/Cornell-VAILab/Raster2Seq>
 28. GitHub - ee-Liu/CAGE, <https://github.com/ee-Liu/CAGE>
 29. dxf-writer - A CDN for npm and GitHub - jsDelivr,
<https://www.jsdelivr.com/package/npm/dxf-writer>
 30. dxfjs/writer: A JavaScript dxf generator written in TypeScript. - GitHub,
<https://github.com/dxfjs/writer>
 31. @tarikjabiri/dxf CDN by jsDelivr - A CDN for npm and GitHub,
<https://www.jsdelivr.com/package/npm/@tarikjabiri/dxf>
 32. dxf-writer - NPM, <https://www.npmjs.com/package/dxf-writer>